



Universidad autónoma de baja california

Ingeniería en computación

Internet de las cosas (IoT)

Practica-lab 11: prototipo IoT (TCP)

Maestro: Dr. Aguilar Noriega Leocundo

Erik garcia Chávez 01275863

Viernes 27 de marzo del 2026

Instrucciones

Usando la plataforma ESP-IDF realizar los ajustes necesarios al prototipo actual para lograr la nueva funcionalidad.

a) El dispositivo IoT prototipo

1. El dispositivo IoT prototipo basado en ESP32, una vez que se enciende deberá conectarse al servidor IoT en la dirección IP (xxxx.xxxx.xxxx.xxx) correspondiente usando el protocolo TCP con puerto 50005. El comando a enviar para la solicitud de acceso tiene el siguiente formato:

UABC:<usuario>:<operación>:<recurso>:<comentario>

donde:

<usuario>: Es un campo que funciona para identificar el ESP32 del usuario y está conformado por a<matrícula>.

<operación>: En este caso de solicitud de acceso la operación estada dada por la letra L (login) o K(keep-alive) una vez que ya se haya logrado el login.

<recurso>: Es un campo que funciona para identificar el recurso a operar, en este caso el S (servidor).

<comentario>: Se refiere a un pequeño texto relacionado a la operación y es opcional.

Ejemplo del comando:

UABC:a12345678:L:S:Login el server

El servidor responderá con un ACK si se logró el acceso, de lo contrario se responde con NACK indicando que no se reconoce la acción solicitada.

Una vez que el ESP32 ha logrado el acceso al servidor se deberá estar enviando cada 10 segundo el siguiente comando con el fin de estar informando al servidor que el ESP32 está activo y conectado. Importante, esto se hace sobre la misma conexión ya establecida.

UABC:<usuario>:K:S:Keep-Alive al server

El servidor responderá con un ACK si se logró la operación, de lo contrario se responde con NACK indicando que no se reconoce la acción solicitada. Si por algún motivo el ESP32 deja de enviar este comando el servidor asumirá que el ESP32 ha quedado fuera de línea, pero si el ESP32 retoma a la actividad se deberá solicitar acceso (login) para luego enviar cada 10 segundo este comando.

b) Recepción de comandos

2. Una vez que el ESP32 logra el punto anterior deberá estar listo para recibir algún comando de operación sobre los recursos del dispositivo mediante alguna operación desde el servidor. Los servicios del dispositivo IoT son el LED, el ADC y PWM.

Comando:

**UABC:<usuario>:<operación>:<recurso>:<valor>:
<comentario>**

Por ejemplo el comando para escribir (operación W) al LED es:

UABC:<usuario>:W:L:1:Encender LED

y si se logró encender el LED la contestación debe ser: ACK:<Estado del LED>, en este caso ACK:1

Para leer (operación R) el estado de LED el comando es:

UABC:<usuario>:R:L: Leer LED

y la contestación es: ACK:<estado del LED>

Para leer el ADC el comando es: UABC:<usuario>:R:A: Lee ADC

y la contestación es: ACK:<valor del ADC>

Nota: La operación W no es permitida para este recurso.

Para leer el PWM el comando es: UABC:<usuario>:R:P: Lee PWM

y la contestación es: ACK:<valor del PWM> -- el valor del PWM es de 0 a 100

Para escribir al PWM el comando es: UABC:<usuario>:W:P: 50:Escribir PWM

y la contestación es: ACK:<valor del PWM>

Si el comando no es reconocido debe retornar un NACK

Desarrollo:

El programa requiere establecer una conexión con un servidor, la forma de realizar esta conexión es por medio de sockets TCP.

El programa hace uso de un protocolo de comunicación entre el servidor y el esp32 cliente, el cual es:

UABC:matricula:R/W:recurso:valor:mensaje

El esp32 tiene con la capacidad de lectura o escritura 3 recursos, **LED**, **PWM**, **ADC**. de los cuales el **ADC** solo puede ser leído.

para lograr esta conexión primero se debemos de crear nuestro socket para establecer la comunicación, este socket como es un protocolo TCP tiene que ser el mismo, es con el que se establece el handshake entre el cliente y el servidor

```
esp_err_t tcp_cliente_init(void){
    tcp_client.connected = 0;
    if (tcp_client.sock >= 0) {
        close(tcp_client.sock);
        tcp_client.sock = -1;
    }

    for(int n_retry = 0 ; n_retry < 5 ; n_retry++){

        ESP_LOGI(TAG, "intento %d de establecer conexion", n_retry);

        tcp_client.sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
        if(tcp_client.sock < 0){
            ESP_LOGE(TAG, "error al crear el descriptor del socket(): %d", errno);
            vTaskDelay(pdMS_TO_TICKS(1000));
            continue;
        }

        struct timeval timeout = { .tv_sec = 5, .tv_usec = 0 };
        setsockopt(tcp_client.sock, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

        struct sockaddr_in server_addr={
            .sin_family = AF_INET,
            .sin_port = htons(tcp_client.host_port),
        };

        inet_pton(AF_INET, tcp_client.host_ip,&server_addr.sin_addr);

        if(connect(tcp_client.sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0){
            ESP_LOGE(TAG, "descriptor <connect(> fallo %d", errno);
            close(tcp_client.sock);
            tcp_client.sock = -1;
            vTaskDelay(pdMS_TO_TICKS(2000));
            continue;
        }

        tcp_client.connected = 1;
        ESP_LOGI(TAG, "conectado a %s:%d", tcp_client.host_ip, tcp_client.host_port);
        return ESP_OK;
    }

    ESP_LOGE(TAG, "no se pudo conectar al servidor en 5 intentos");
    tcp_client.connected = 0;
    return ESP_FAIL;
}
```

Esta función crea el archivo descriptor del socket e intenten establecer la conexión al menos 5 veces, si en esas cinco veces no se pudo establecer la conexión el programa sale y se mantiene en espera que el usuario quiera volver a intentar establecer la conexión con las mismas credenciales una vez mas o desea cambiar las credenciales.

Para este tipo de conexión es importante el mantener la misma conexión, una conexión UDP no usa la misma sesión para comunicarse y recibir información, cada vez crea una nueva, pero con TCP esta sesión se mantiene en toda la comunicación hasta que se cierre por alguna de las dos partes.

Para esto se estableció una estructura que contendrá información relevante de la conexión TCP:

```

1
2  typedef struct {
3      char *host_ip;
4      uint16_t host_port;
5      int sock;
6      int connected;
7      int logged_in;
8
9  }tcp_client_t;
10

```

En esta estructura nos funcionara para guardar el descriptor del socket, así como la información sobre la conexión, si estamos logeados si se logró hacer el login al server y la ip y puerto a la que se debe de conectar.

Para esto se utilizan comandos que el esp recibe por medio de UART, los comandos principales son:

SSI:<nom_red> PSWD:<contrasena> -> para establecer una nueva red

HOST_IP:<ip_hot> HOST_PORT:<port> -> para establecer un nuevo host

La tarea responsable de esto es la tarea llamada: **task_cmd_uart** la cual va a recibir por medio de una cola lo que se ingresó por UART,

Al parsear la cadena podremos saber que comandos fueron los que se ingresaron y comparamos, los comandos deben de estar en el orden para que funcionen en, en otro caso daría un error:

```

if(strcmp(tipo,"SSID") ==0 && tokens[1] != NULL){

    char *ssid = strchr(tokens[0], ':');
    char *pswd = strchr(tokens[1], ':');

    if(ssid == NULL || pswd == NULL){
        ESP_LOGE(TAG,"formato incorrecto. Usa: SSID:<nombre> PSWD:<password>");
        goto cleanup;
    }

    ssid++;
    pswd++;
    ret = update_setup_cred(ssid, pswd, NULL, "SSID");
    if(ret !=ESP_FAIL){
        xEventGroupSetBits(s_wifi_event_group, WIFI_UPDATE); //evento para WIFI
        xEventGroupSetBits(g_tcp_event_group, BREAK_UPDATE_WIFI); //evento para TCP -
    }
    else{

        ESP_LOGE(TAG, "no se pudieron guardar las credenciales WIFI");

    }
}
}

```

En esta comparación verifica si es la red wifi la que se debe de actualizar si es así, por grupo de bits indicamos ese cambio y si hay una conexión tcp activa se cierra, y procedemos a actualizar la red wifi y volver a establecer la conexión.

Pero para cambiar TCP no es necesario desconectarnos del wifi, cuando se quiere cambiar de host de igual forma tiene que cerrar las conexiones ya abiertas:



```
else if(strcmp(tipo, "HOST_IP") == 0 && tokens[1] != NULL){

    char *host_ip = strchr(tokens[0], ':');
    char *port = strchr(tokens[1], ':');

    if (host_ip == NULL || port == NULL) {
        ESP_LOGE(TAG, "formato incorrecto. Usa: HOST_IP:<ip> PORT:<puerto>");
        goto cleanup;
    }

    host_ip++;
    port++;

    ret = update_setup_cred(host_ip, port, NULL, "HOST_IP");

    if(ret != ESP_FAIL){
        xEventGroupSetBits(g_tcp_event_group, UPDATE_TCP);
    }
    else{
        ESP_LOGE(TAG, "nERROR: no se pudo guardar el nuevo servidor");
    }
}

}
```

La misma lógica se estaría aplicando para actualizar las credenciales del socket TCP.

Para poder establecer una interacción entre el esp y el servidor antes se debió iniciar sesión al servidor.

La siguiente función inicia todos los pasos para poder establecer la comunicación, la función tiene un ciclo infinito porque va a iniciar varios procesos en distintos tiempos, primero la función manda a llamar a la función que crea el descriptor del socket como primer paso, primero debe de poder establecer una conexión con ese servidor,

Como segundo paso se debe de iniciar sesión en el servicio del servido, mediante el siguiente comando, **UABC:a1275863:L:S:Login en el server**

Se tiene una función que se encarga de construir la cadena ascii y enviar por medio de la función **send()** al servidor, el esp espera un **ACK** lo que indica que se logró iniciar sesión satisfactoriamente

Antes de enviar los **keep-alives**, así como iniciar las tareas que reciben del servidor y la que procesa la petición, primero debió de haber iniciado sesión de manera correcta.

```

void setup_tcp(void){
    while(1){
        //creando el socket TCP
        esp_err_t ret = tcp_cliente_init();

        static int n_login=0;
        int len;

        if(ret == ESP_OK){
            uart_write_bytes(UART_MAIN, UART_GREEN, strlen(UART_GREEN));
            const char *m = "\r\nconexion con el servidor establecida\r\n";
            uart_write_bytes(UART_MAIN, m, strlen(m));
            uart_write_bytes(UART_MAIN, UART_RESET, strlen(UART_RESET));

            if(tcp_client.logged_in != 1){
                char n_rety_log[64];
                len= sprintf(n_rety_log, sizeof(n_rety_log), "intento #%i\r\n", n_login);
                uart_write_bytes(global_uart.NUM_PORT, n_rety_log, len);
                //no ha iniciado sesion.
                send_info.op = OP_LOGIN;

                ret = send_message(&tcp_client, &send_info);
                //verificamos que se vuelva a intentar el login
                if(ret != ESP_OK){
                    //no se pudo conectar
                    char err[80];
                    len = sprintf(err, sizeof(err), "\r\nno se pudo hacer login al servidor, revise si esta arriba\r\n ");
                    uart_write_bytes(global_uart.NUM_PORT, err, len);
                    continue;
                }

                //se logro hacer el login
                tcp_client.logged_in = 1;
            }
        }
    }
}

```

Cuando se inicia sesión en el miembro de login de la estructura se establece en 1. Cuando esto pasa entonces se inician 3 tareas, se inician en este momento, porque estarían queriendo enviar al servidor keep alives cuando aún no se ha iniciado sesión, se podría, pero no es lo correcto, el servidor no podrá acceder a los recursos del esp

La tarea **recv_task** recibe las peticiones del servidor, solo realizara esto, recibe, limpia la cadena y envía por medio de una cola a la tarea **tcp_process_task** que se encargó de leer lo que el servidor respondió o pide y decide que acción, que servicio necesita proceder.



```

1
2  static bool ka_created = false;
3  if (!ka_created) {
4      xTaskCreate(keep_alive_task, "keep_alive_task", 2048, NULL, 6, NULL);
5      xTaskCreate(recv_task, "recv_task", 4098, NULL, 8, NULL);
6      xTaskCreate(tcp_process_task, "tcp_process_task", 4098, NULL, 8, NULL);
7      ka_created = true;
8  }
9

```

```

// limpiar bits relevantes antes de esperar
xEventGroupClearBits(g_tcp_event_group, RETRY_SERVER | UPDATE_TCP | BREAK_UPDATE_WIFI | NO_RETRY_TCP);
EventBits_t bits = xEventGroupWaitBits(g_tcp_event_group, RETRY_SERVER | UPDATE_TCP | BREAK_UPDATE_WIFI | NO_RETRY_TCP, pdTRUE, pdFALSE, portMAX_DELAY);

if (bits & NO_RETRY_TCP) {
    uart_write_bytes(UART_MAIN, UART_RED, strlen(UART_RED));
    const char *bye = "\r\nterminando... reiniciando ESP\r\n";
    uart_write_bytes(UART_MAIN, bye, strlen(bye));
    uart_write_bytes(UART_MAIN, UART_RESET, strlen(UART_RESET));
    vTaskDelay(pdMS_TO_TICKS(1000));
    esp_restart();
}

if (bits & BREAK_UPDATE_WIFI) {
    // cerrar socket si habia uno abierto
    if (tcp_client.sock >= 0) {
        close(tcp_client.sock);
        tcp_client.sock = -1;
    }
    tcp_client.connected = 0;

    wifi_reconnect();
}

```

La función se queda en espera de actualización de wifi o TCP en cualquiera que pase, reiniciara y reconectara el esp o vuelve a conectar con las nuevas direcciones del servidor TCP.

Para enviar los paquetes al servidor se utiliza la función

send_message(tcp_client_t *sockfd, send_info_t *msg);, esta función ya que los datos que serán enviados están cargados a la estructura **send_info_t** es cuando se manda a llamar a esta función.

La estructura contiene la siguiente información:

```

typedef struct{
    op_type_t op;
    uint16_t value;
    char comment[MAX_COMMENT_LEN];
}send_info_t;

```

El ESP solo debe de contestar **ACK:<value>** cuando se recibió la trama de manera correcta, o **NACK** cuando no se entendió, por lo que necesita un enum indicando que tipo de mensaje enviara un **ACK, NACK, LOGIN o KEEP ALIVE**, el valor que enviara de regreso y el comentario funciona para **login y keep alive**.

La función de **send_message()** construye el frame ASCII que será enviado por TCP, decidiendo dependiendo de que tipo de mensaje es y que va a responder el ESP cliente al servidor.

Cada vez que se necite enviar información al servidor como el eco de la petición del servidor, y si no entendido la petición o el formato está mal o el inicio de sesión se manda a llamar a esta función, que solo manda ese mensaje por el servidor.


```

esp_err_t send_message(tcp_client_t *sockfd, send_info_t *msg){
    char buffer[128];
    int len=0;
    switch(msg->op){
        case OP_LOGIN : {
            len = snprintf(buffer, sizeof(buffer), "UABC:%s:L:S:Login server\n", user);
            }break;

        case OP_ACK: {
            len = snprintf(buffer, sizeof(buffer), "ACK:%d\n", msg->value);
            }break;
        case OP_NACK :{
            len = snprintf(buffer, sizeof(buffer), "NACK\n");
            }break;

        default: {
            ESP_LOGI(TAG, "debugger");
            }break;
    }

    //verificamos que el snprintf no se haya psado
    if(len < 0 || len>=(int)sizeof(buffer)){
        const char *err= "\r\nbuffer overflow al construir mensaje\r\n";
        uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
        uart_write_bytes(global_uart.NUM_PORT, err, strlen(err));
        uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));

        return ESP_FAIL;
    }

    int sent = send(sockfd->sock, buffer, len, 0);

    if(sent < 0){
        char err_str[32];
        snprintf(err_str, sizeof(err_str), "send() fallo errno: %d\r\n", errno);
        uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
        uart_write_bytes(global_uart.NUM_PORT, err_str, strlen(err_str));
        uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
        return ESP_FAIL;
    }
    uart_write_bytes(global_uart.NUM_PORT, UART_GREEN, strlen(UART_GREEN));
    const char *info_send = "dato enviado: ";
    uart_write_bytes(global_uart.NUM_PORT, info_send, strlen(info_send));
    uart_write_bytes(global_uart.NUM_PORT, buffer, strlen(buffer));
    uart_write_bytes(global_uart.NUM_PORT, "\r\n", 2);
    uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));

    return ESP_OK;
}

```

La tarea de **keep_alive()** es una tarea aparte porque está mandando cada 10 segundos un keep alive por la misma conexión, el mismo socket establecido entre el servidor y el esp32.

```

void keep_alive_task(void *params){

    while(1){

        //debemos de construir lo que vamos a mandar

        char payload[50];

        char *inicio = "UABC";
        char *final = "K:S:Keep-Alive al server\n";

        snprintf(payload, sizeof(payload), "%s:%s:%s", inicio, user, final);

        int err = send(tcp_client.sock, payload, strlen(payload), 0);

        if(err < 0 ){

            char err[80];
            snprintf(err, sizeof(err), "\r\nocurrió un error al enviar la informacion errno: %d\r\n", errno);
            //ocurrio un error, no pudo enviar la ifnromacion la servidor
            uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
            uart_write_bytes(global_uart.NUM_PORT, err, strlen(err));
            uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
        }
        else{
            const char *success = "keep alive enviado\r\n";
            uart_write_bytes(global_uart.NUM_PORT, UART_GREEN, strlen(UART_GREEN));
            uart_write_bytes(global_uart.NUM_PORT, success, strlen(success));
            uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
        }

        //esperamos 10 segundos
        vTaskDelay(pdMS_TO_TICKS(10000));
    }
}

```

La tarea **recv_task** se encarga solamente de cachear lo que el servidor envió hacia el esp32, limpiar la trama y mandarlo hacia la tarea **tcp_proccess_task()** que es en donde se mira hacia dentro de la trama y verifica que la trama este bien construida y si lo está que es lo que pide, si lo que pide esta entre los parámetros adecuados entonces contestamos con un **ACK** si no lo está entonces se contesta con un **NACK**

Tarea recv_task():

Primero se verifica que se hasta recibido algo, si hubo un error en la recepción de hace saber al usuario

```
1 void recv_task(void *params){
2
3     char rx_buffer[128];
4     static char *line_buffer = NULL;
5     static size_t line_len = 0;
6     while(1){
7
8         int len = recv(tcp_client.sock,rx_buffer, sizeof(rx_buffer)-1,0);
9
10        if(len < 0 ){
11
12            if(errno == EAGAIN || errno == EWOULDBLOCK){
13                vTaskDelay(pdMS_TO_TICKS(50));
14                continue; // timeout normal del SO_RCVTIMEO, no es error
15            }
16            //hubon un error al recibir del servidor
17            char err[80];
18            sprintf(err, sizeof(err),"\r\nTCP_LIB: recv fallo errno: %d\r\n", errno);
19
20            uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
21            uart_write_bytes(global_uart.NUM_PORT,err,strlen(err));
22            uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
```

En el caso que si se pudo recibir la trama del servidor, terminamos la liana con un carácter nulo, para saber en dónde termina, se procede a limpiar la trama como quitando los salto de línea o el retorno de carro, colocamos el contenido en un buffer temporal con el cual se creara una copia en un nuevo espacio de memoria del contenido recibido, lo enviamos por medio de una cola a la tarea

recv_proccess_task() en donde se toma la decisión de que es lo que procede dependiendo de lo el frame contenga.

```

1 }else if(len > 0){
2
3     //hubo exito al recibir del servidor.
4     // mostramos un debug de la informacion que recibimos del cliente
5     rx_buffer[len]='\0';
6
7     for(int i=0; i< len ;i++){
8
9         char c = rx_buffer[i];
10
11         if(c == '\n'){
12             //linea completa
13             if(line_buffer !=NULL && line_len >0 ){
14                 //eliminamos \r
15                 if(line_buffer[line_len-1] == '\r'){
16                     line_buffer[line_len-1] = '\0';
17                 }
18                 else{
19                     line_buffer[line_len] = '\0';
20                 }
21             }
22             //creamos la copia para ser enviada
23             char *msg_copy = strdup(line_buffer);
24             if(msg_copy){
25                 //echo de la informacion que llego
26                 uart_write_bytes(global_uart.NUM_PORT, UART_CYAN,    strlen(UART_CYAN));
27                 const char *prefix = "\r\nRECV : ";
28                 uart_write_bytes(global_uart.NUM_PORT, prefix, strlen(prefix));
29                 uart_write_bytes(global_uart.NUM_PORT, msg_copy, strlen(msg_copy));
30                 uart_write_bytes(global_uart.NUM_PORT, "\r\n", 2);
31                 uart_write_bytes(global_uart.NUM_PORT, UART_RESET,    strlen(UART_RESET));
32                 //encolar el puntero, metodo no bloqueante
33                 if(xQueueSend(tcp_rx_queue, &msg_copy, 0) != pdTRUE){
34                     free(msg_copy);
35                     char err[80];
36                     snprintf(err, sizeof(err),"\r\nTCP_LIB: cola llena, mensaje destarcado -> errno: %d\r\n", errno);
37
38                     uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
39                     uart_write_bytes(global_uart.NUM_PORT,err,strlen(err));
40                     uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
41                 }
42             }
43             //liberar memoria
44             free(line_buffer);
45             line_buffer=NULL;
46             line_len=0;
47         }
48         else{
49             //acumular caracter
50             char *new_buf = realloc(line_buffer, line_len +2);
51             if(new_buf){
52                 line_buffer = new_buf;
53                 line_buffer[line_len++]=c;
54             }
55             else{
56                 //error de memoria
57                 free(line_buffer);
58                 line_buffer=NULL;
59                 line_len=0;
60             }
61         }
62     }
63 }
64 }

```

En el caso que lo que se recibió fue 0 entonces el servidor esta cerrado, por lo que se cierra la conexión del lado del esp32.

```

1 else if(len == 0){
2     //conexion cerrada
3     char err[80];
4     snprintf(err, sizeof(err),"\r\nTCP_LIB: el servidor cerro la conexion -> errno: %d \r\n", errno);
5     uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
6     uart_write_bytes(global_uart.NUM_PORT,err,strlen(err));
7     uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
8     tcp_client.connected = 0;
9     tcp_client.logged_in =0;
10    close(tcp_client.sock);
11    tcp_client.sock =-1;
12
13 }
14
15 vTaskDelay(pdMS_TO_TICKS(50));
16 }

```

Para poder realizar la verificación de estableció una estructura con los elementos que debe de contener la trama, se inicializo con las posibles peticiones que se pueden solicitar al esp.

```
1 typedef struct{
2     char *header; //UABC
3     char *user; //user que ya tenemos
4     char resource[3]; //[ "L", "A", "P" ]
5     char operation[2]; //[ 'R', 'W' ]
6     int value; //debemos de verificar que el valor sea un valor adecuado, tipo numero no caracter
7
8 }format_request_t;
9
10 extern format_request_t format_request;
```

Esta estructura la inicializamos con sus posibles valores:

```
format_request.header = "UABC";
format_request.resource[0] = 'L'; //LED : R and W
format_request.resource[1] = 'A'; //ADC : only R
format_request.resource[2] = 'P'; // PWD : R nad W

format_request.operation[0] = 'R'; // read
format_request.operation[1] = 'W'; //wriute

format_request.user = user;
```

Lo primero seria verificar si llego un **ACK** o un **NACK** que es lo más sencillo, el servidor consta eso cuando llega un algo que el esp manda.

```
void tcp_process_task(void *params){

    char *msg;
    char **tokens;
    static int mem_request;
    int modulo =
    char buffer[80];

    esp_err_t ret;

    while(1){

        //recibira los datos por cola
        if(xQueueReceive(tcp_rx_queue,&msg, portMAX_DELAY)){

            //el mas sencilllo, verificamos con ACK o en NACK del servidor.
            if(strncmp(msg, "ACK", 3) == 0){
                int len = snprintf(buffer, sizeof(buffer), "\r\nMAIN: ACK recibido -> %s\r\n", msg);
                uart_write_bytes(global_uart.NUM_PORT, UART_GREEN, strlen(UART_GREEN));
                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
                free(msg);
                continue; // no parsear, esperar siguiente mensaje
            }

            if(strncmp(msg, "NACK", 4) == 0){
                int len = snprintf(buffer, sizeof(buffer), "\r\nMAIN: NACK recibido -> %s\r\n", msg);
                uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
                free(msg);
                continue;
            }

        }

    }

}
```

Después si no es el caso entonces quiere decir que se recibió un comando, por lo que se pasa a parsear la cadena para separar cada comando en un token el cual puede ser verificado con mayor facilidad:

```
mem_request=0;
tokens=parse_input_rcv(msg);

//verificamos que se haya parseado correctamente
if(tokens == NULL || tokens[0] == NULL){
    free(msg);
    continue;
}
char current_op = 0; // guarda 'R' o 'W' del case 2 para usarlo en case 3
char *aux;
```

En cada proceso se tiene que verificar que el formato este correctamente estructurado como el header que sería **UABC**, después si el dispositivo está en algún **broadcast** verifica que el mensaje sea para el usuario registrado en el dispositivo, si no es así manda un **NACK**, pero indica por consola que no es para el usuario registrado.

Después verifica que el tercer token tenga la operación, la opción que se requiere hacer si **READ** o **WRITE**:

```
for(mem_request = 0; tokens[mem_request] != NULL; mem_request++){
    aux = tokens[mem_request];

    switch(mem_request){

        case 0: // "UABC"
            if(strcmp(aux, format_request.header) != 0){
                send_info.op = OP_NACK;
                send_message(&tcp_client, &send_info);
                goto cleanup;
            }
            break;

        case 1: // usuario
            if(strcmp(aux, user) != 0){
                int len = snprintf(buffer, sizeof(buffer), "\r\nN- MAIN: peticion no para: %s\r\n", format_request.user);
                uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
                goto cleanup;
            }
            break;

        case 2: // operacion R o W
            current_op = *aux;
            if(current_op != 'R' && current_op != 'W'){
                send_info.op = OP_NACK;
                send_message(&tcp_client, &send_info);
                goto cleanup;
            }
            break;
    }
}
```

Listamos los recursos disponibles para su lectura y escritura, indicando que un ADC no puede ser escrito solo leído.

```
case 3: // recurso L, A, P
    if(current_op == 'R'){
        if(*aux == 'L'){
            send_info.op = OP_ACK;
            send_info.value = led_state;
            send_message(&tcp_client, &send_info);
        }
        else if(*aux == 'A'){
            send_info.op = OP_ACK;
            send_info.value = read_adc(ADC_CHANNEL);
            send_message(&tcp_client, &send_info);
        }
        else if(*aux == 'P'){
            send_info.op = OP_ACK;
            send_info.value = pwm_get_duty();
            send_message(&tcp_client, &send_info);
        }
        else{
            send_info.op = OP_NACK;
            send_message(&tcp_client, &send_info);
            goto cleanup;
        }
    }
    else{ // W
        if(*aux == 'A'){ // ADC no se puede escribir
            send_info.op = OP_NACK;
            send_message(&tcp_client, &send_info);
            goto cleanup;
        }
        // L y P necesitan el valor del case 4, lo guardamos
        // solo validamos que el recurso sea válido
        if(*aux != 'L' && *aux != 'P'){
            send_info.op = OP_NACK;
            send_message(&tcp_client, &send_info);
            goto cleanup;
        }
    }
    break;
```

Si la opción fue escritura, se escribe en el led o el adc el valor enviado desde el servidor

```

case 4: // valor (solo para W)
    if(current_op == 'W'){
        char recurso = *tokens[3];
        if(recurso == 'L'){
            uint8_t val;
            if(validate_binary(aux, &val) == ESP_OK){
                led_state = val;
                gpio_set_level(OUTPUT_PIN, led_state);
                send_info.op = OP_ACK;
                send_info.value = val;
                send_message(&tcp_client, &send_info);
            } else {
                send_info.op = OP_NACK;
                send_message(&tcp_client, &send_info);
                goto cleanup;
            }
        }
        else if(recurso == 'P'){
            uint16_t val;
            if(validate_pwm(aux, &val) == ESP_OK){
                pwm_set_duty(val);
                send_info.op = OP_ACK;
                send_info.value = val;
                send_message(&tcp_client, &send_info);
            } else {
                send_info.op = OP_NACK;
                send_message(&tcp_client, &send_info);
                goto cleanup;
            }
        }
    }
    break;

```

Al final de cada proceso de la trama limpiamos la memoria

```

//al ultimo liberamos la memots
cleanup:
//liberamos tokens
for (int i = 0; tokens[i] != NULL; i++) {
    free(tokens[i]);
}
free(tokens);
free(msg);

```

Al poner a correr el programa, el programa inicia sesión y manda keep alives cada 10s


```
conexion con el servidor establecida
intento #0
dato enviado: UABC:a1275863:L:S:Login server

keep alive enviado

RECV  : ACK:Login OK

MAIN: ACK recibido -> ACK:Login OK

RECV  : ACK:Keep-Alive OK

MAIN: ACK recibido -> ACK:Keep-Alive OK
keep alive enviado

RECV  : ACK:Keep-Alive OK

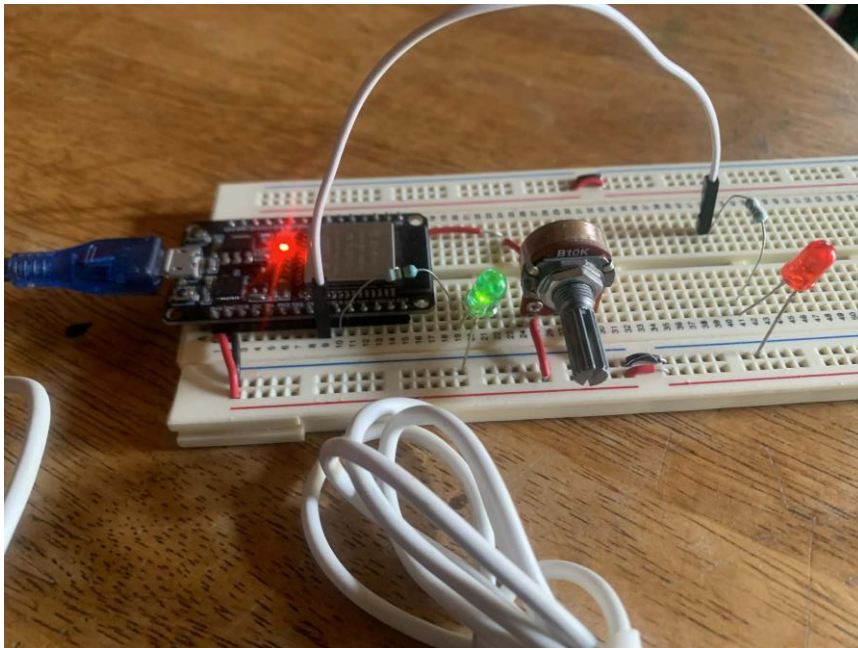
MAIN: ACK recibido -> ACK:Keep-Alive OK
```

Si se solicita que se prenda el led, lo que se recibe es:

```
MAIN: ACK recibido -> ACK:Keep-Alive OK

RECV  : UABC:a1275863:W:L:1:Encender LED
dato enviado: ACK:1
```

El led prende y en cuando prende debe de contestar **ACK:1** lo cual se esta logrando hacer.



Al solicitar leer ADC el esp32 consta con el valor que tiene actualmente el potenciómetro:

```
RECV : UABC:a1275863:R:A::Leer ADC  
dato enviado: ACK:3007
```

El servidor pide escribir al pwm 50% por lo que es una luz muy tenue la que se puede percibir en el led:

```
RECV : UABC:a1275863:W:P:50:Escribir PWM  
dato enviado: ACK:50
```

Por lo que, si ahora el servido pide leer, el esp debe de constar con ese 50% que contiene el pwm:

```
RECV : UABC:a1275863:R:P::Leer PWM  
dato enviado: ACK:50
```

Circuito armado:

